

6CS030: Big Data

Workshop - 8

Full Name : Narayan Lohani

Student Number : 2408869

Group : L6CG15

Tutor : Sushil Timilsina

Submission Date : April 30, 2026

Table of Contents

- SECTION 1 3
- SECTION 2 9
- SECTION 3:..... 10
 - CHECKING CLUSTER HEALTH 11
 - Exercise To do..... 11
 - CREATING A RESOURCE 14
 - LISTING INDICES 14
 - ADDING DATA TO RESOURCE 15
 - Exercise To Do:..... 16
 - FETCHING THE RECORD 17
 - Exercise to do 18

SECTION 1

1. What is ELK stack? Explain it using its architecture.

ELT stack is a set of three open source tools: Elasticsearch, Logstash and Kibana used together to take data from any source, in any format, in one place (centralized logging), and then perform search, analyzation or visualization it in real time.

Brief explanation of each component

1. **Elasticsearch:** A distributed, RESTful search and analytics engine built on Apache Lucene used for storing data in a scalable way, index them for fast search, and support even complex analytics.
2. **Logstash:** A server-side data processing pipeline that takes input data from multiple sources, transforms it, and stores it to Elasticsearch (or other destinations).
3. **Kibana:** A web-based visualization and exploration interface for data stored in Elasticsearch. It is used for creating dashboards, graphs, and performing queries.

Architecture explanation

Logstash receives different kind of data (logs, metrics, events) from different platforms (Applications, S3 buckets, HTTP requests). Then, logstash with need parses, transforms or filters data (example: converting messy data into structured JSON) and forward the clean data to store in Elasticsearch.

Elasticsearch then indexes the data in order to make the searching faster, and stores them.

Kibana queries Elasticsearch when user through its UI search, visualize, or analyze data.

Currently, Beats which are lightweight data shippers are deployed on servers to ship data to Logstash or in some cases directly to Elasticsearch.

2. What is Shard in ElasticSearch?

Shards are horizontal partition of data in an index. They are the fundamental unit of scalability in Elasticsearch.

Let's understand through analogy.

If Index were to be a large book that is too large for a single person to hold, we can divide the book into different chapters that are easy to hold. These small chapters are shards.

Formal definition wise, a shard is a self-contained Apache Lucene index that holds a subset of the data of a parent.

Shards allow use to split data across multiple nodes making scalability way easier. Using shards we can run queries in parallel, making searches faster.

Types:

- **Primary shards:** Each document stored in Elasticsearch belongs to exactly one primary shard, which is responsible for all write operations (index, update, delete).
- **Replica shards:** Copies of primary shards for redundancy, fault tolerance, and read scalability. Replicas are placed on different nodes from their primaries.

Benefits:

- **Horizontal scaling:** By splitting an index into multiple shards and distributing them across different nodes in a cluster, Elasticsearch can store more amount of data than a single server can hold and can parallelize query execution.
- **High availability:** If a node fails, replicas can be promoted to primaries, ensuring data is not lost when on node fail.
- **Performance:** Queries can be run in parallel across different nodes and as smaller shards are faster to process, data can be searched quicker.

3. What are the three major components of Logstash?

The three major components of Logstash are: **Input**, **Filter** and **Output**.

- **Input:** This component is responsible for collecting data from different sources such as files, beats, kafka, syslog, jdbc, etc.
- **Filter:** This component then “processes” the data by parsing, transforming and enriching events.
- **Output:** Ships the processed data forward either to Elasticsearch or maybe different destinations.

Filter uses different components for data processing.

It uses:

- Grok to convert unstructured data to structured data.
- Mutate to rename/remove fields from data.
- Geoip to add location data.
- Drop to discard events.

4. Perform a case study on the possible usage of ELK stack by any one of the following companies leveraging big data.

a. Netflix

Netflix is a leading example of how the ELK stack can be leveraged within a big-data ecosystem to drive observability, security, and user-experience improvements at global scale. With over 200 million subscribers and tens of thousands of microservices, Netflix generates trillions of events every day, including logs, streaming telemetry, and user-interaction data. To make sense of this flood of information, Netflix relies on Elasticsearch-based search and analytics alongside streaming pipelines such as Kafka and Keystone, effectively integrating ELK-style capabilities into its cloud-native architecture.

ELK is used extensively at Netflix for operational monitoring and troubleshooting. Logs from backend services, edge devices, and streaming clients are ingested into Elasticsearch through tools similar to Logstash or Kafka-based shippers, then visualized in Kibana-like dashboards. When users in a specific region start experiencing buffering or playback errors, engineers can query logs across millions of devices in near real time, correlate incidents with CDN and microservice metrics, and quickly isolate faulty components or misconfigurations. This dramatically reduces mean time to detect and resolve incidents, improving platform reliability and uptime.

In addition to operations, Netflix uses its ELK-style infrastructure for security monitoring and anomaly detection. Network and authentication logs are indexed into Elasticsearch, enabling teams to search for brute-force attempts, suspicious IP ranges, or unusual traffic patterns across regions. Because Elasticsearch supports fast aggregations and faceted queries, security analysts can run ad-hoc threat-hunting workloads over petabyte-scale log histories without waiting for batch reports, strengthening proactive and reactive security postures.

Netflix also leverages ELK-powered analytics for customer-experience and UI insights. Application and UI logs capture how users interact with features such as play, pause, search, and recommendation clicks allowing product teams to spot subtle issues that affect only a small fraction of users. For example, if a new UI button fails for 0.1% of viewers, Elasticsearch-backed dashboards can surface this “needle in a haystack” before the problem becomes a trending topic on social media, enabling rapid fixes and smoother rollouts.

Under the hood, Netflix runs hundreds of Elasticsearch nodes across many clusters, consuming terabytes to petabytes of log and event data. The company uses Raigad, an open-source sidecar manager, to automate cluster discovery, backups, node health checks, and failover on AWS, which reduces operational overhead at scale. Elasticsearch is tightly integrated with Kafka-driven streaming pipelines and batch layers such as Spark and Hadoop on S3, so both real-time and historical data are searchable and analyzable in a single stack.

The business impact of this ELK-based approach is clear. Netflix achieves faster incident response, deeper system observability, stronger security posture, and higher-quality user experiences, all while operating in a highly distributed, cloud-native environment. By combining Elasticsearch search with big-data pipelines, Netflix not only improves reliability but also informs recommendation and search-relevance models through logged user behavior, illustrating how the ELK stack can become a central pillar of modern, data-driven platforms.

5. Explore Fuzzy Search in ELK.

Fuzzy search is a feature implemented within Elasticsearch that allows users to find documents containing terms that are similar to but not exactly the same as the query term. It is especially useful for handling typos or spelling variations.

How It Works

1. **Edit distance:** Fuzzy queries use the **Levenshtein edit distance algorithm** to measure similarity. The edit distance counts the minimal number of single-character changes needed to turn one string into another.

Four types of edits are considered:

- Changing a character (box → fox)
 - Removing a character (black → lack)
 - Inserting a character (sic → sick)
 - Transposing two adjacent characters (act → cat)
2. **Variation expansion:** For a given search term and a specified edit distance (fuzziness), the fuzzy query automatically generates all possible terms that are within that distance. It then performs an exact match against each variant.
 3. **Scoring:** Documents with a smaller edit distance (more similar) are ranked higher in the relevance score.

SECTION 2

For this, you need to install the ELK stack in your machine.

Elastic Running:

```
[narayan@narayan ~ % /opt/homebrew/opt/elasticsearch-full/bin/elasticsearch
warning: usage of JAVA_HOME is deprecated, use ES_JAVA_HOME
warning: usage of JAVA_HOME is deprecated, use ES_JAVA_HOME
Apr 24, 2024 7:28:51 AM sun.util.locale.provider.LocaleProviderAdapter <clinit>
WARNING: COMPAT locale provider will be removed in a future release
[2026-04-24T07:28:52.495][INFO ][o.e.n.Node               ] [narayan.local] version[7.17.4], pid[51421], build[default/tar/79878662c54c886ae89286c685d9f1051a9d6411/2022-05-18T18:04:28.964345128Z], OS[Mac
OS X/24.4.1/arm64t802], JVM[Homebrew/OpenJDK 64-Bit Server VM/21.0.10/21.0.10]
[2026-04-24T07:28:52.496][INFO ][o.e.n.Node               ] [narayan.local] JVM home [/opt/homebrew/Cellar/openjdk@21/21.0.10/libexec/openjdk.jdk/Contents/Home], using bundled JDK [false]
[2026-04-24T07:28:52.496][INFO ][o.e.n.Node               ] [narayan.local] JVM arguments [-Xshare:auto, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -
Xsslm, -Djava.awt.headless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -XX:-OmitStackTraceInFastThrow, -XX:+ShowCodeDetailsInExceptionMessages, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimization
=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog.j2.shutdownHookEnabled=false, -Dlog.j2.disable.jmx=true, -Dlog.j2.formatMsgNoLookups=true, -Djava.locale.pro
viders=SPT,COMPAT, --add-opens=java.base/java.io=ALL-UNNAMED, -Djava.security.manager=allow, -XX:+UseG1GC, -Djava.io.tmpdir=/var/folders/X3/7mw598m3c7mnp5jnzprw88000n/7/elasticsearch-44437140837864428
37, -XX:+HeapDumpOnOutOfMemoryError, -XX:+ExitOnOutOfMemoryError, -XX:HeapDumpPath=data, -XX:ErrorFile=logs/hs_err_pid%p.log, -Xlog:gc*,gc+age=trace,safepoint:file=/opt/homebrew/var/log/elasticsearch/gc.l
og:utctime,pid,tags:filecount=32,filesize=64m, -Xms8192m, -Xmx8192m, -XX:MaxDirectMemorySize=4294967296, -XX:InitiatingHeapOccupancyPercent=30, -XX:G1ReservePercent=25, -Des.path.home=/opt/homebrew/Cellar
/elasticsearch-full/7.17.4/libexec, -Des.path.conf=/opt/homebrew/etc/elasticsearch, -Des.distribution.flavor=default, -Des.distribution.type=tar, -Des.bundled_jdk=true]
[2026-04-24T07:28:53.384][INFO ][o.e.p.PluginsService     ] [narayan.local] loaded module [analysis-common]
[2026-04-24T07:28:53.384][INFO ][o.e.p.PluginsService     ] [narayan.local] loaded module [analysis-common]
```

Kibana Running:

```
[narayan@narayan ~ % /opt/homebrew/opt/kibana-full/bin/kibana
log [07:25:43.175] [info][plugins-service] Plugin 'metricsEntities' is disabled.
log [07:25:43.222] [info][server][Preboot][http] http server running at http://localhost:5601
log [07:25:43.264] [warning][config][deprecation] Environment variable 'KIBANA_PATH_CONF' is deprecated. It has been replaced with 'KBN_PATH_CONF' pointing to a config folder
log [07:25:43.266] [warning][config][deprecation] Environment variable 'DATA_PATH' will be removed. It has been replaced with kibana.yml setting 'path.data'
log [07:25:43.266] [warning][config][deprecation] Starting in 8.0, the kibana logging format will be changing. This may affect you if you are doing any special handling of your Kibana logs, such as in
gesting logs into Elasticsearch for further analysis. If you are using the new logging configuration, you are already receiving logs in both old and new formats, and the old format will simply be going aw
ay. If you are not yet using the new logging configuration, the log format will change upon upgrade to 8.0. Beginning in 8.0, the format of JSON logs will be ECS-compatible JSON, and the default pattern l
og format will be configurable with our new logging system. Please refer to the documentation for more information about the new logging format.
log [07:25:43.267] [warning][config][deprecation] The default mechanism for Reporting privileges will work differently in future versions, which will affect the behavior of this cluster. Set 'xpack.re
porting.roles.enabled' to 'false' to adopt the future behavior before upgrading.
log [07:25:43.268] [warning][config][deprecation] User sessions will automatically time out after 8 hours of inactivity starting in 8.0. Override this value to change the timeout.
log [07:25:43.269] [warning][config][deprecation] Users are automatically required to log in again after 30 days starting in 8.0. Override this value to change the timeout.
log [07:25:43.328] [info][plugins-system][standard] Setting up 1131 plugins: [translation-licenses,globalSearch,globalSearchProviders,features-licenses,guard-core,usageCollection,packagingTool]
```

LogStash Running:

```
[narayan@narayan ~ % /opt/logstash/bin/logstash -f /opt/logstash/config/logstash-simple.conf
Using JAVA_HOME defined java: /opt/homebrew/Cellar/openjdk@21/21.0.10/libexec/openjdk.jdk/Contents/Home
WARNING: Using JAVA_HOME while Logstash distribution comes with a bundled JDK.
DEPRECATION: The use of JAVA_HOME is now deprecated and will be removed starting from 8.0. Please configure LS_JAVA_HOME instead.
Sending Logstash logs to /opt/logstash/logs which is now configured via log.j2.properties
[2026-04-24T07:48:26.886][INFO ][logstash.runner           ] Log4j configuration path used is: /opt/logstash/config/log4j2.properties
[2026-04-24T07:48:26.841][INFO ][logstash.runner           ] Starting Logstash ("logstash.version">="7.17.4", "jruby.version">="jruby 9.2.20.1 (2.8.0) 2021-11-30 2a29d2fbd1 OpenJDK 64-Bit Server VM 21.0.10
on 21.0.10 +indy +jit [darwin-aarch64]")
[2026-04-24T07:48:26.842][INFO ][logstash.runner           ] JVM bootstrap flags: [-Xms1g, -Xmx1g, -Djava.awt.headless=true, -Dfile.encoding=UTF-8, -Djdk.io.File.enableADS=true, -Djruby.compile.invokedynam
ic=true, -Djruby.jit.threshold=0, -Djruby.ragexp.interruptible=true, -XX:+HeapDumpOnOutOfMemoryError, -Djava.security.epdfile=/dev/urandom, -Dlog.j2.isThreadContextMapInheritable=true, --add-opens=java.b
ase/sun.nio.ch=ALL-UNNAMED, --add-opens=java.base/java.io=ALL-UNNAMED]
[2026-04-24T07:48:26.848][INFO ][logstash.settings         ] Creating directory (:settings=>"path.queue", :path=>"/opt/logstash/data/queue")
[2026-04-24T07:48:26.851][INFO ][logstash.settings         ] Creating directory (:settings=>"path.dead_letter_queue", :path=>"/opt/logstash/data/dead_letter_queue")
[2026-04-24T07:48:26.867][WARN ][logstash.config.source.multilocal] Ignoring the 'pipelines.yml' file because modules or command line options are specified
[2026-04-24T07:48:26.873][INFO ][logstash.agent           ] No persistent UUID file found. Generating new UUID (:uuid=>"8b07ab58-b7ce-4e3c-9f39-687b973cd86", :path=>"/opt/logstash/data/uuid")
[2026-04-24T07:48:26.958][ERROR][logstash.instrument.periodicpoller.jvm] Unknown garbage collector name (:name=>"G1 Concurrent GC")
[2026-04-24T07:48:27.153][INFO ][logstash.runner           ] Successfully started Logstash! endpoint {:port=>5601, :url=>http://localhost:5601}
```

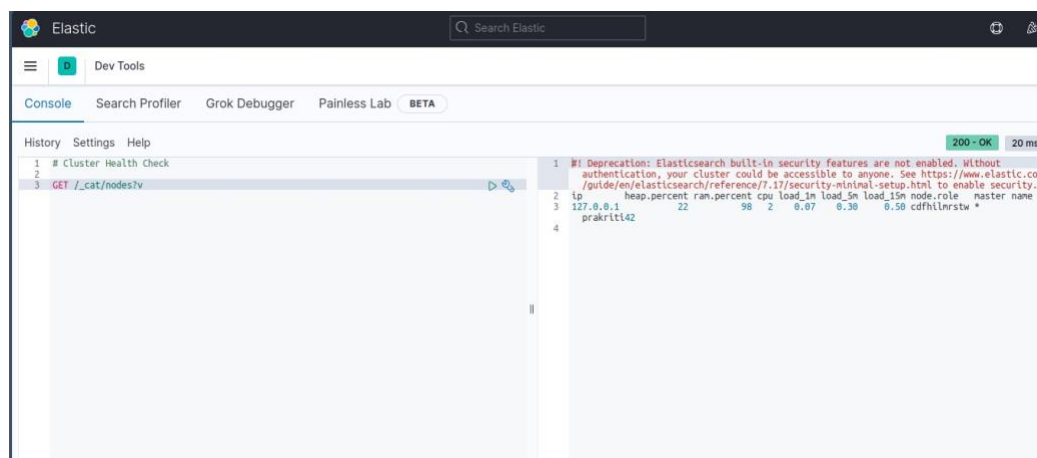
SECTION 3:

As part of building primary familiarity with ELK lets try exploring the provided elastic apis for data manipulation using the Kibana interface.

To start with this, open up Kibana on a web browser and go to

`http://localhost:5601/app/dev_tools#/console`

this should lead you to the Console that will allow you to kick start the task. The GUI looks like the following.



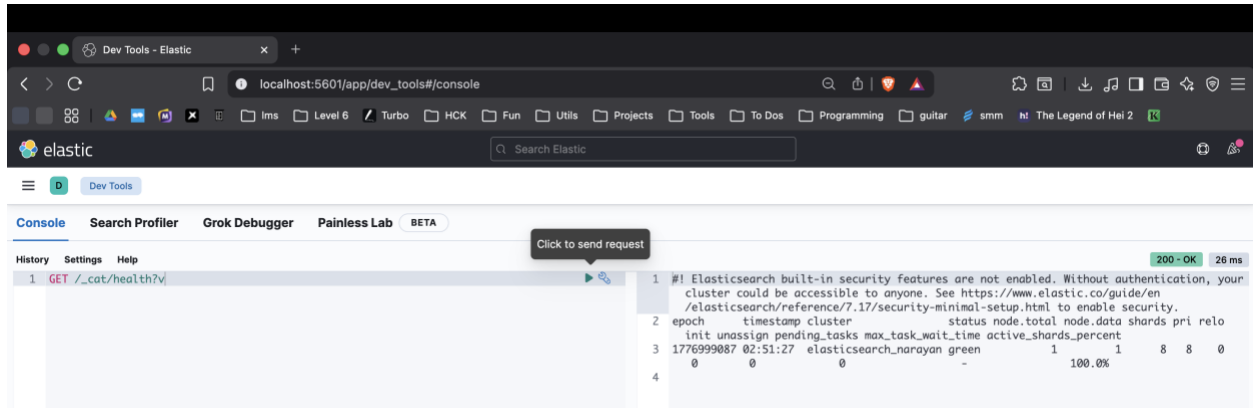
Make sure you have elasticsearch enabled too.

Checking Cluster Health

Your task first is to try out a way to make a cluster health check via an Elastic API call.

Consider trying the command

GET /_cat/health?v



Exercise To do

1. Observe and write down your understanding of the result. Also, try sending a call request without the ?v, note down the difference if present.

Making sense of the response:

Epoch

Field	Value	Description
epoch	1776999087	Unix timestamp, machine time format
timestamp	02:51:27	Human-readable time of the snapshot
cluster	elasticsearch_narayan	Name of the Elasticsearch cluster
status	green	Overall health status (green = fully healthy)
node.total	1	Total number of nodes in the cluster
node.data	1	Number of nodes that store data
shards	8	Total number of shards (primary + replicas)
pri	8	Number of primary shards (original data holders)

relo	0	Shards currently relocating between nodes
init	0	Shards currently initializing (starting up/recovering)
unassign	0	Shards not assigned to any node (problem if > 0)
pending_tasks	0	Number of cluster tasks waiting to be executed
max_task_wait_time	-	Longest wait time of pending tasks (“-” = none waiting)
active_shards_percent	100.0%	Percentage of shards that are active and working

The difference between having ‘v’ and not having ‘v’ at the end of request is to include the labels (“epoch”, ... , “active_shards_percent”) along with the values while displaying the health status results. Excluding ‘v’ displays the values only.

Including v:

```
1 #! Elasticsearch built-in security features are not enabled. Without authentication, your cluster could be accessible to anyone. See https://www.elastic.co
2 /guide/en/elasticsearch/reference/7.17/security-minimal-setup.html to enable security.
3 epoch timestamp cluster status node.total node.data shards pri relo init unassign pending_tasks max_task_wait_time active_shards_percent
4 1776999978 03:06:18 elasticsearch_narayan green 1 1 8 8 0 0 0 0 0 0 - 100.0%
```

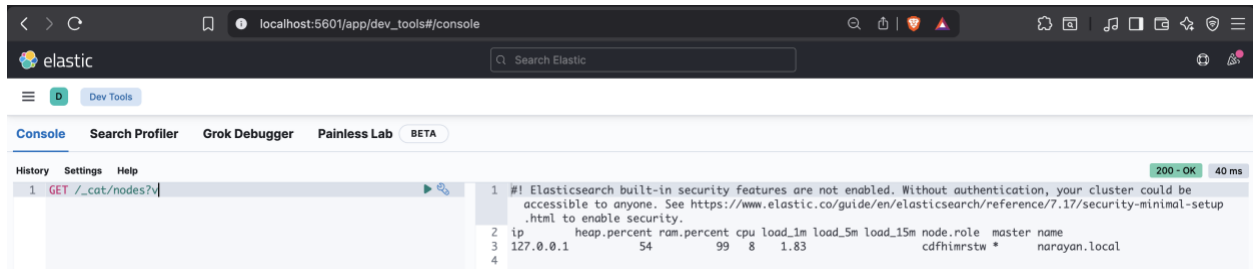
Excluding v:

```
1 #! Elasticsearch built-in security features are not enabled. Without authentication, your cluster could be accessible to anyone. See https://www.elastic.co
2 /guide/en/elasticsearch/reference/7.17/security-minimal-setup.html to enable security.
3 17769999616 03:00:16 elasticsearch_narayan green 1 1 8 8 0 0 0 0 - 100.0%
```

2. Make an api call to check the node health.

Node health can be checked using following command:

GET /_cat/nodes?v

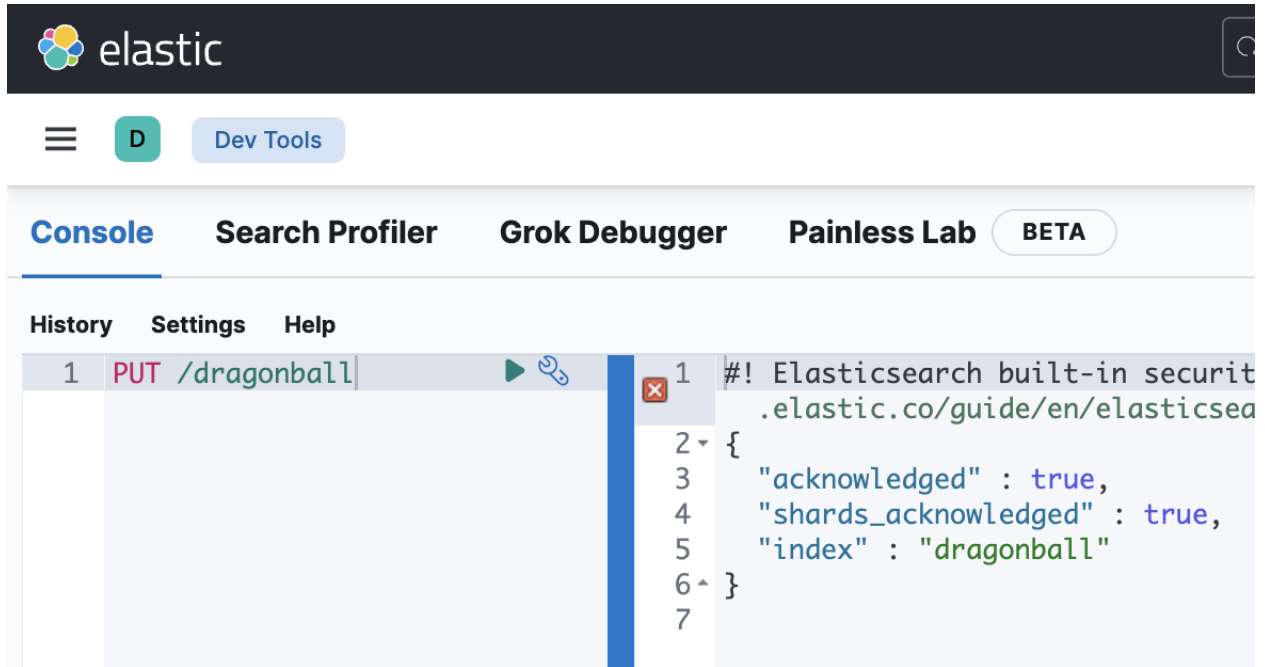


Creating a Resource

Now, lets try adding in data to our nosql database which in current case is ElasticSearch.

You can try creating a resource using the command

PUT /resourcename



The screenshot shows the Elastic UI Console. The 'Console' tab is selected. The command bar shows a PUT request to /dragonball. The response is a JSON object indicating the index was created successfully.

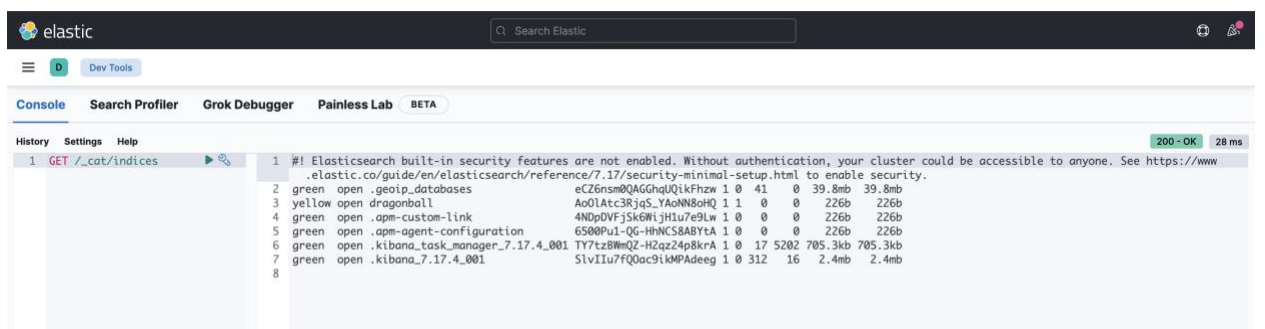
```
1 PUT /dragonball
```

```
1 {
2   "acknowledged" : true,
3   "shards_acknowledged" : true,
4   "index" : "dragonball"
5 }
6
7
```

Listing Indices

Now, try out the list indices call to observe any changes in the resources using,

GET /_cat/indices



The screenshot shows the Elastic UI Console. The 'Console' tab is selected. The command bar shows a GET request to /_cat/indices. The response is a table listing the indices in the cluster.

```
1 GET /_cat/indices
```

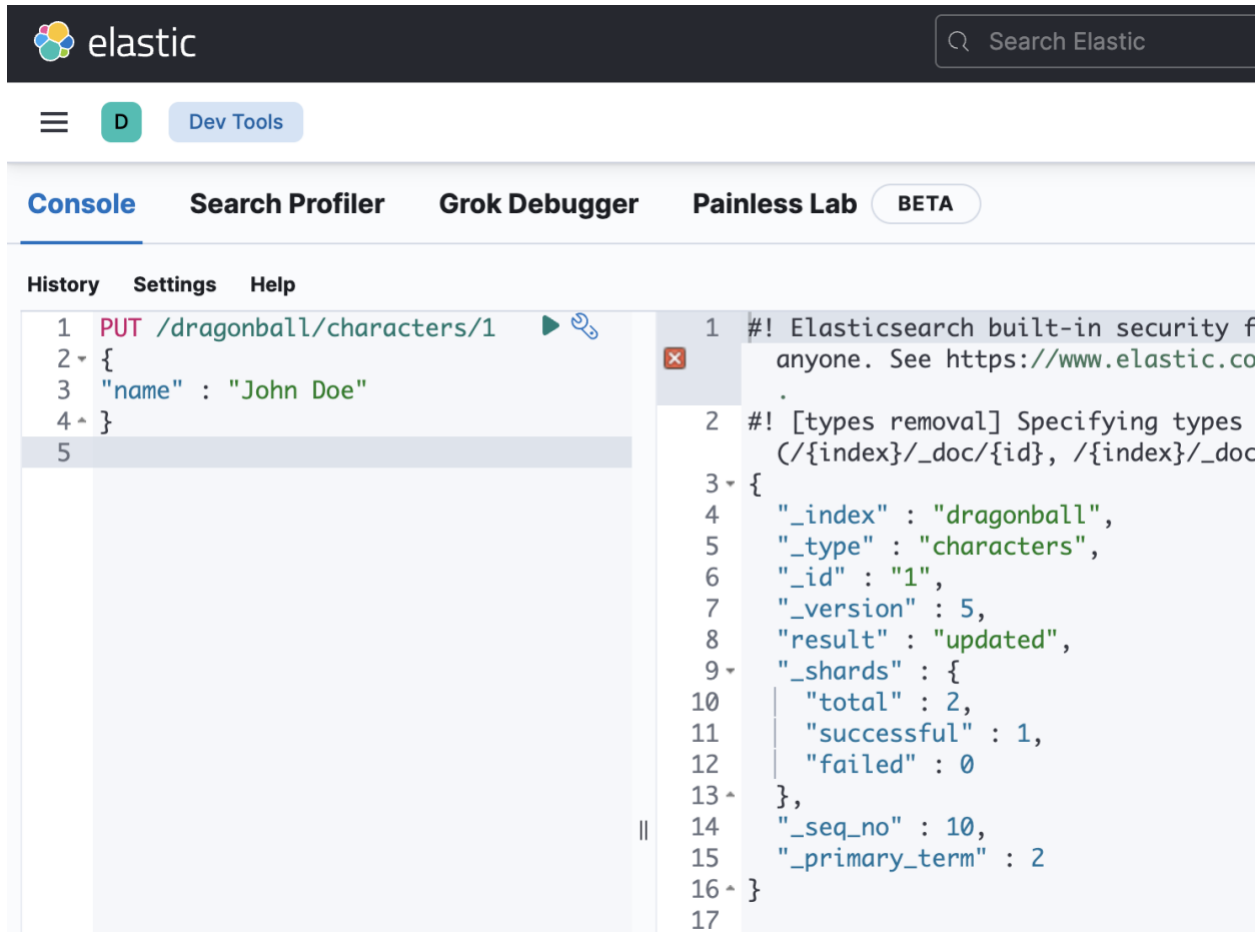
```
1 {
2   "indices" : [
3     {
4       "index" : ".geoip_databases",
5       "status" : "green",
6       "type" : "open",
7       "uuid" : "ecZ6nsn0QAGhaUQlkfhzw",
8       "generation" : 1,
9       "primary_shards" : 41,
10      "replica_shards" : 0,
11      "size_in_bytes" : 39860,
12      "format_size_in_bytes" : 39860
13     },
14     {
15       "index" : "dragonball",
16       "status" : "yellow",
17       "type" : "open",
18       "uuid" : "Ao01Atc3RjaS_YAoNN8oHQ",
19       "generation" : 1,
20       "primary_shards" : 1,
21       "replica_shards" : 0,
22       "size_in_bytes" : 226,
23       "format_size_in_bytes" : 226
24     },
25     {
26       "index" : ".apm-custom-link",
27       "status" : "green",
28       "type" : "open",
29       "uuid" : "4NDpDVFjSk6Wi_jH1u7e9Lw",
30       "generation" : 1,
31       "primary_shards" : 0,
32       "replica_shards" : 0,
33       "size_in_bytes" : 226,
34       "format_size_in_bytes" : 226
35     },
36     {
37       "index" : ".apm-agent-configuration",
38       "status" : "green",
39       "type" : "open",
40       "uuid" : "6500Pu1-QG-HhNCS8ABYtA",
41       "generation" : 1,
42       "primary_shards" : 0,
43       "replica_shards" : 0,
44       "size_in_bytes" : 226,
45       "format_size_in_bytes" : 226
46     },
47     {
48       "index" : ".kibana_task_manager_7.17.4_001",
49       "status" : "green",
50       "type" : "open",
51       "uuid" : "TY7tz8WmQZ-H2qz24p8krA",
52       "generation" : 1,
53       "primary_shards" : 17,
54       "replica_shards" : 0,
55       "size_in_bytes" : 5202705,
56       "format_size_in_bytes" : 705300
57     },
58     {
59       "index" : ".kibana_7.17.4_001",
60       "status" : "green",
61       "type" : "open",
62       "uuid" : "S1vIIu7fQ0ac9ikMPAdeeg",
63       "generation" : 1,
64       "primary_shards" : 312,
65       "replica_shards" : 16,
66       "size_in_bytes" : 2400000,
67       "format_size_in_bytes" : 2400000
68     }
69   ]
70 }
```

Next on, let's move ahead with addition of data into the resource we just created,

Adding Data to Resource

PUT /resourcename/colname/1

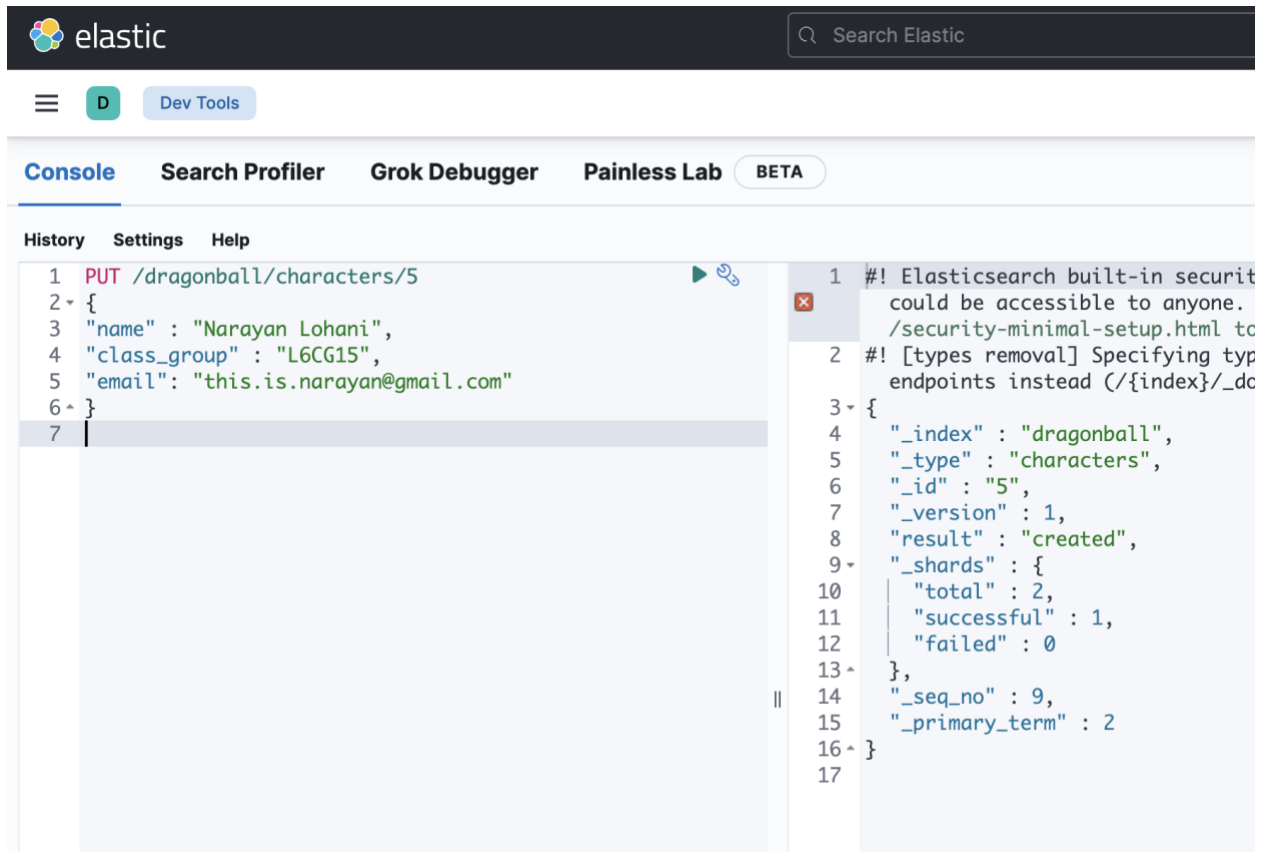
```
{  
  "name" : "John Doe",  
}
```



Note: I had already inserted four data in same document 1 before “John Doe”, thus is the reason for version 5.

Exercise To Do:

I helped you add John Doe, its your turn to add in your personal data (consider populating it with your name, class group and email) to the same resource.



The screenshot shows the Elastic Dev Tools interface. The top bar includes the Elastic logo, a search bar, and a 'Dev Tools' button. Below this is a navigation bar with 'Console', 'Search Profiler', 'Grok Debugger', 'Painless Lab', and a 'BETA' badge. The 'Console' tab is active, showing a 'History' section with 'Settings' and 'Help' links. The main area displays a REST client request and response. The request is a PUT to /dragonball/characters/5 with a JSON body containing name, class_group, and email. The response is a JSON object with metadata and the created character data.

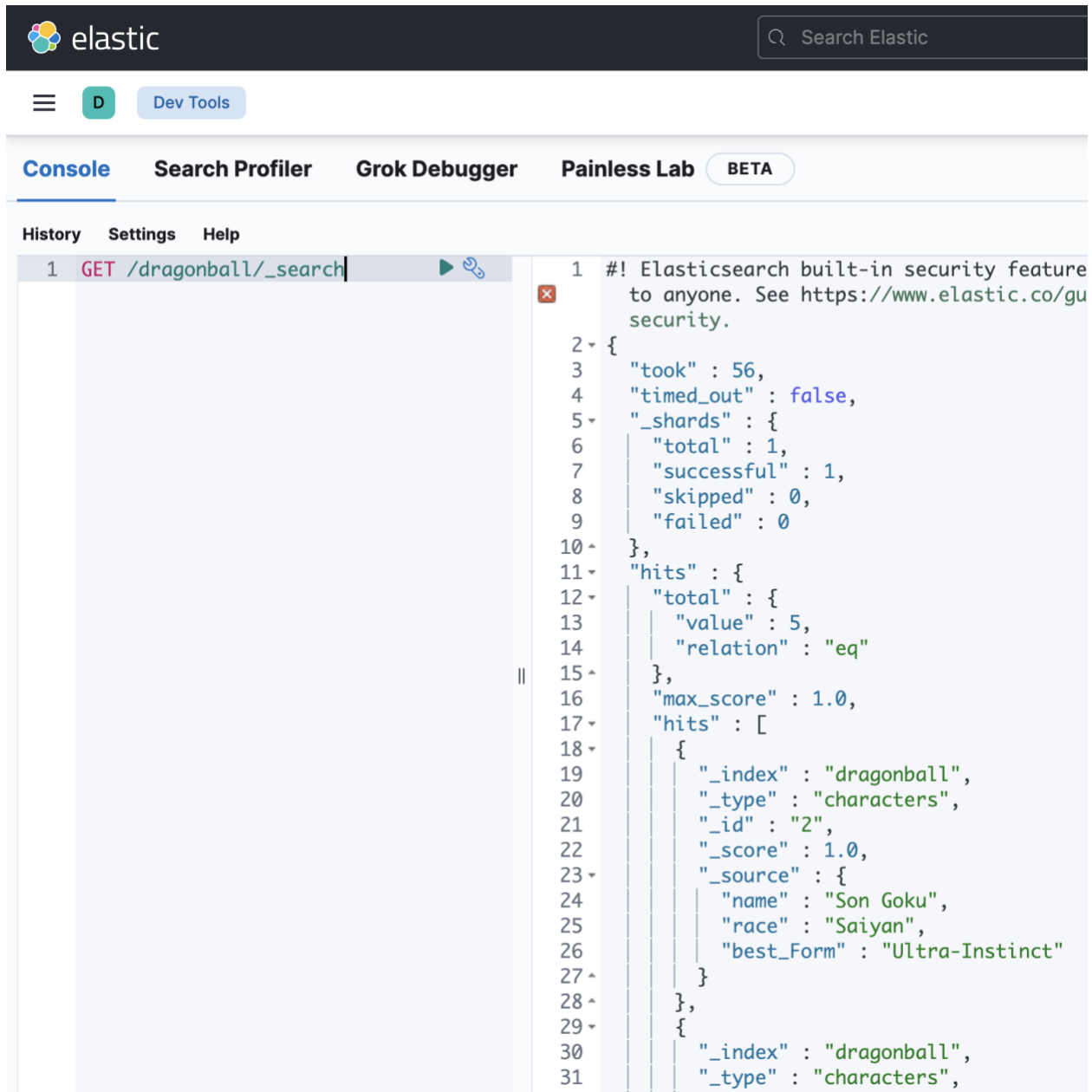
```
1 PUT /dragonball/characters/5
2 {
3   "name" : "Narayan Lohani",
4   "class_group" : "L6CG15",
5   "email": "this.is.narayan@gmail.com"
6 }
7
```

```
1 1 #! Elasticsearch built-in security
2 2 #! [types removal] Specifying type
3 3 {
4 4   "_index" : "dragonball",
5 5   "_type" : "characters",
6 6   "_id" : "5",
7 7   "_version" : 1,
8 8   "result" : "created",
9 9   "_shards" : {
10 10     "total" : 2,
11 11     "successful" : 1,
12 12     "failed" : 0
13 13   },
14 14   "_seq_no" : 9,
15 15   "_primary_term" : 2
16 16 }
17
```


Fetching the Record

In order to query you can try

GET /resourcename/_search



The screenshot shows the Elastic Dev Tools interface. At the top, the Elastic logo and a search bar are visible. Below the navigation bar, the 'Console' tab is selected. The console shows a single request: a GET request to /dragonball/_search. The response is a JSON object indicating a successful search with 5 hits. The first hit is for a character named Son Goku, who is a Saiyan and has the Ultra-Instinct form.

```
1 GET /dragonball/_search
```

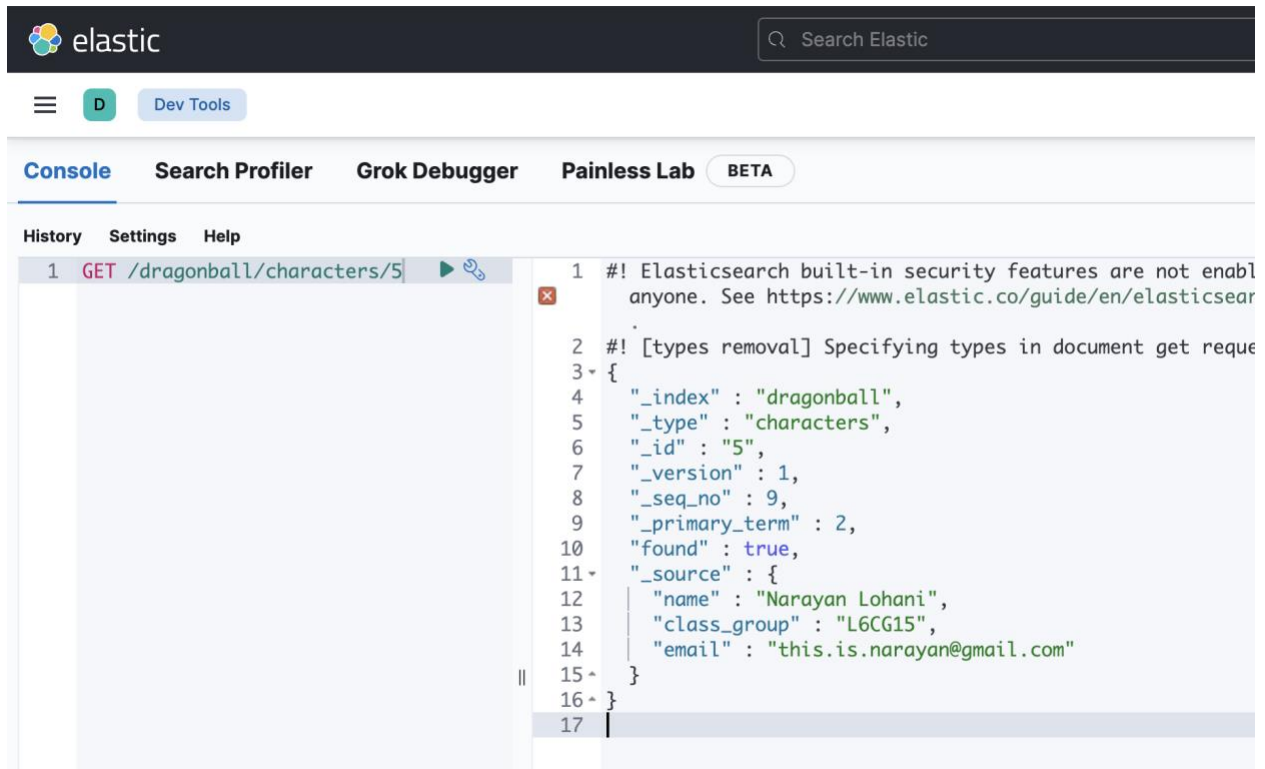
```
1  #! Elasticsearch built-in security feature
2  to anyone. See https://www.elastic.co/gu
3  security.
4  {
5    "took" : 56,
6    "timed_out" : false,
7    "_shards" : {
8      "total" : 1,
9      "successful" : 1,
10     "skipped" : 0,
11     "failed" : 0
12   },
13   "hits" : {
14     "total" : {
15       "value" : 5,
16       "relation" : "eq"
17     },
18     "max_score" : 1.0,
19     "hits" : [
20       {
21         "_index" : "dragonball",
22         "_type" : "characters",
23         "_id" : "2",
24         "_score" : 1.0,
25         "_source" : {
26           "name" : "Son Goku",
27           "race" : "Saiyan",
28           "best_Form" : "Ultra-Instinct"
29         }
30       },
31       {
32         "_index" : "dragonball",
33         "_type" : "characters",
```

Exercise to do

1. Write an api query call to fetch the second data record (that was your personal details).

Because I had inserted my data at `/dragonball/character/5`, the document id became 5.

Thus fetching the document with id 5 (which is my personal details).



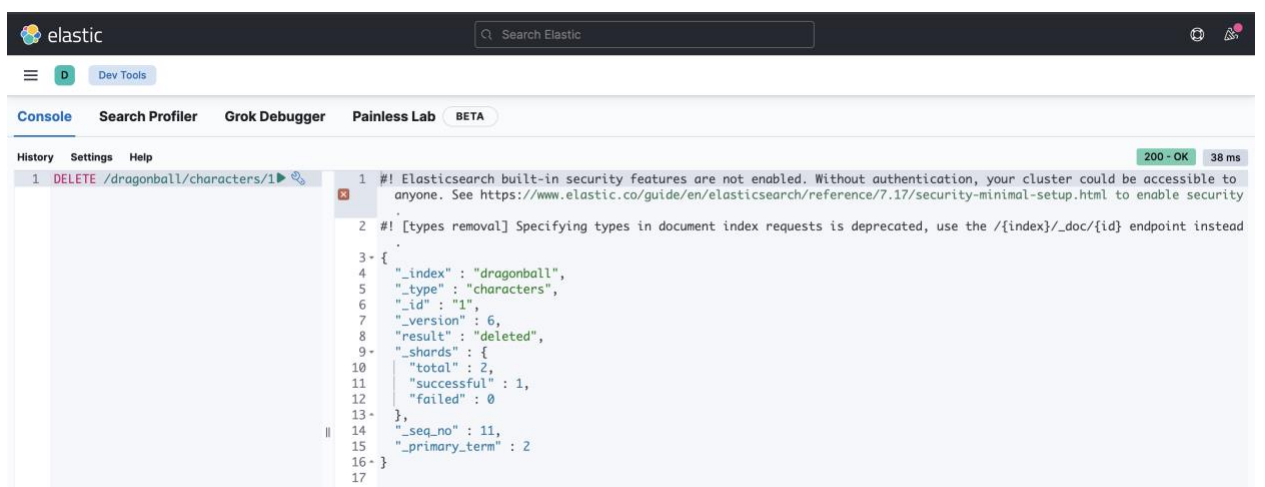
The screenshot shows the Elastic Dev Tools interface. The 'Console' tab is active, displaying a GET request to `/dragonball/characters/5`. The response is a JSON object representing a character record.

```
1 GET /dragonball/characters/5
```

```
1 200 OK 38 ms
2 {
3   "_index": "dragonball",
4   "_type": "characters",
5   "_id": "5",
6   "_version": 1,
7   "_seq_no": 9,
8   "_primary_term": 2,
9   "found": true,
10  "_source": {
11    "name": "Narayan Lohani",
12    "class_group": "L6CG15",
13    "email": "this.is.narayan@gmail.com"
14  }
15 }
16 }
```

2. Write a query to remove the data record of John Doe from the resource.

John Doe data was inserted at `_id : 1`, so deleting that document will delete the record.



The screenshot shows the Elastic Dev Tools interface. The 'Console' tab is active, displaying a DELETE request to `/dragonball/characters/1`. The response is a JSON object indicating the successful deletion of the document.

```
1 DELETE /dragonball/characters/1
```

```
1 200 - OK 38 ms
2 {
3   "_index": "dragonball",
4   "_type": "characters",
5   "_id": "1",
6   "_version": 6,
7   "result": "deleted",
8   "_shards": {
9     "total": 2,
10    "successful": 1,
11    "failed": 0
12  },
13  "_seq_no": 11,
14  "_primary_term": 2
15 }
16 }
```